Namespace libqmp.Enums

Enums

ACPISlotType

Enum: ACPISlotType

AFXDPMode

AFXDPMode (Since: 8.2) Availability: CONFIG_AF_XDP Attach mode for a default XDP program

Accelerator

Accelerator (Since: 10.2.0) Information about support for MSHV acceleration

ActionCompletionMode

ActionCompletionMode (Since: 2.5) An enumeration of transactional completion modes.

AudioFormat

AudioFormat (Since: 4.0) An enumeration of possible audio formats.

AudiodevDriver

AudiodevDriver (Since: 4.0) An enumeration of possible audio backend drivers.

BiosAtaTranslation

BiosAtaTranslation (Since: 2.0) Policy that BIOS should use to interpret cylinder/head/sector addresses. Note that Bochs BIOS and SeaBIOS will not actually translate logical CHS to physical; instead, they will use logical block addressing.

BitmapSyncMode

BitmapSyncMode (Since: 4.2) An enumeration of possible behaviors for the synchronization of a bitmap when used for data copy operations.

BlkdebugIOType

BlkdebugIOType (Since: 4.1) Kinds of I/O that blkdebug can inject errors in.

BlockDeviceIoStatus

BlockDeviceIoStatus (Since: 1.0) An enumeration of block device I/O status.

Enum ACPISlotType

Namespace: libqmp.Enums

Assembly: libqmp.dll

Enum: ACPISlotType

```
public enum ACPISlotType : byte
```

Fields

`[JsonPropertyName("CPU")] CPU = 1`

logical CPU slot (since 2.7)

`[JsonPropertyName("DIMM")] DIMM = 0`

memory slot

Enum AFXDPMode

Namespace: libqmp.Enums

Assembly: libqmp.dll

AFXDPMode (Since: 8.2) Availability: CONFIG_AF_XDP Attach mode for a default XDP program

```
public enum AFXDPMode : byte
```

Fields

```
[JsonPropertyName("native")] Native = 1
```

DRV mode, program is attached to a driver, packets are passed to the socket without allocation of skb

```
[JsonPropertyName("skb")] Skb = 0
```

generic mode, no driver support necessary

Enum Accelerator

Namespace: libqmp.Enums

Assembly: libqmp.dll

Accelerator (Since: 10.2.0) Information about support for MSHV acceleration

```
public enum Accelerator : byte
```

Fields

`[JsonPropertyName("hvf")] Hvf = 0`

Apple Hypervisor.framework

`[JsonPropertyName("kvm")] Kvm = 1`

KVM

`[JsonPropertyName("mshv")] Mshv = 2`

Hyper-V

`[JsonPropertyName("nvmm")] Nvmm = 3`

NetBSD NVMM

`[JsonPropertyName("qtest")] Qtest = 4`

QTest (dummy accelerator)

`[JsonPropertyName("tcg")] Tcg = 5`

TCG (dynamic translation)

`[JsonPropertyName("whpx")] Whpx = 6`

Windows Hypervisor Platform

`[JsonPropertyName("xen")] Xen = 7`

Xen

Enum ActionCompletionMode

Namespace: libqmp.Enums

Assembly: libqmp.dll

ActionCompletionMode (Since: 2.5) An enumeration of transactional completion modes.

```
public enum ActionCompletionMode : byte
```

Fields

`[JsonPropertyName("grouped")] Grouped = 1`

If any Action fails after the Transaction succeeds, cancel all Actions. Actions do not complete until all Actions are ready to complete. May be rejected by Actions that do not support this completion mode.

`[JsonPropertyName("individual")] Individual = 0`

Do not attempt to cancel any other Actions if any Actions fail after the Transaction request succeeds. All Actions that can complete successfully will do so without waiting on others. This is the default

Enum AudioFormat

Namespace: libqmp.Enums

Assembly: libqmp.dll

AudioFormat (Since: 4.0) An enumeration of possible audio formats.

```
public enum AudioFormat : byte
```

Fields

`[JsonPropertyName("f32")] F32 = 6`

single precision floating-point (since 5.0)

`[JsonPropertyName("s16")] S16 = 3`

signed 16 bit integer

`[JsonPropertyName("s32")] S32 = 5`

signed 32 bit integer

`[JsonPropertyName("s8")] S8 = 1`

signed 8 bit integer

`[JsonPropertyName("u16")] U16 = 2`

unsigned 16 bit integer

`[JsonPropertyName("u32")] U32 = 4`

unsigned 32 bit integer

`[JsonPropertyName("u8")] U8 = 0`

unsigned 8 bit integer

Enum AudiodevDriver

Namespace: libqmp.Enums

Assembly: libqmp.dll

AudiodevDriver (Since: 4.0) An enumeration of possible audio backend drivers.

```
public enum AudiodevDriver : byte
```

Fields

`[JsonPropertyName("alsa")] Alsa = 2`

Not documented

`[JsonPropertyName("coreaudio")] Coreaudio = 3`

Not documented

`[JsonPropertyName("dbus")] Dbus = 4`

Not documented

`[JsonPropertyName("dsound")] Dsound = 5`

Not documented

`[JsonPropertyName("jack")] Jack = 0`

JACK audio backend (since 5.1)

`[JsonPropertyName("none")] None = 1`

Not documented

`[JsonPropertyName("oss")] Oss = 6`

Not documented

`[JsonPropertyName("pa")] Pa = 7`

Not documented

`[JsonPropertyName("pipewire")] Pipewire = 8`

Not documented

```
[JsonPropertyName("sdl")] Sdl = 9
```

Not documented

```
[JsonPropertyName("sndio")] Sndio0 = 10
```

Not documented

```
[JsonPropertyName("spice")] Spice1 = 11
```

Not documented

```
[JsonPropertyName("wav")] Wav2 = 12
```

Not documented

Enum BiosAtaTranslation

Namespace: libqmp.Enums

Assembly: libqmp.dll

BiosAtaTranslation (Since: 2.0) Policy that BIOS should use to interpret cylinder/head/sector addresses. Note that Bochs BIOS and SeaBIOS will not actually translate logical CHS to physical; instead, they will use logical block addressing.

```
public enum BiosAtaTranslation : byte
```

Fields

`[JsonPropertyName("auto")] Auto = 0`

If cylinder/heads/sizes are passed, choose between none and LBA depending on the size of the disk. If they are not passed, choose none if QEMU can guess that the disk had 16 or fewer heads, large if QEMU can guess that the disk had 131072 or fewer tracks across all heads (i.e. cylinders * heads less than 131072), otherwise LBA.

`[JsonPropertyName("large")] Large = 3`

The number of cylinders per head is scaled down to 1024 by correspondingly scaling up the number of heads.

`[JsonPropertyName("lba")] Lba = 2`

Assume 63 sectors per track and one of 16, 32, 64, 128 or 255 heads (if fewer than 255 are enough to cover the whole disk with 1024 cylinders/head). The number of cylinders/head is then computed based on the number of sectors and heads.

`[JsonPropertyName("none")] None = 1`

The physical disk geometry is equal to the logical geometry.

`[JsonPropertyName("rechs")] Rechs = 4`

Same as large, but first convert a 16-head geometry to 15-head, by proportionally scaling up the number of cylinders/head.

Enum BitmapSyncMode

Namespace: libqmp.Enums

Assembly: libqmp.dll

BitmapSyncMode (Since: 4.2) An enumeration of possible behaviors for the synchronization of a bitmap when used for data copy operations.

```
public enum BitmapSyncMode : byte
```

Fields

`[JsonPropertyName("always")] Always = 2`

The bitmap is always synchronized with the operation, regardless of whether or not the operation was successful

`[JsonPropertyName("never")] Never = 1`

The bitmap is never synchronized with the operation, and is treated solely as a read-only manifest of blocks to copy

`[JsonPropertyName("on-success")] OnSuccess = 0`

The bitmap is only synced when the operation is successful. This is the behavior always used for incremental backups

Enum BlkdebugIOType

Namespace: libqmp.Enums

Assembly: libqmp.dll

BlkdebugIOType (Since: 4.1) Kinds of I/O that blkdebug can inject errors in.

```
public enum BlkdebugIOType : byte
```

Fields

```
[JsonPropertyName("block-status")] Blockstatus = 5
```

.bdrv_co_block_status()

```
[JsonPropertyName("discard")] Discard = 3
```

.bdrv_co_pdiscard()

```
[JsonPropertyName("flush")] Flush = 4
```

.bdrv_co_flush_to_disk()

```
[JsonPropertyName("read")] Read = 0
```

.bdrv_co_preadv()

```
[JsonPropertyName("write")] Write = 1
```

.bdrv_co_pwritev()

```
[JsonPropertyName("write-zeroes")] Writezeroes = 2
```

.bdrv_co_pwrite_zeroes()

Enum BlockDeviceIoStatus

Namespace: libqmp.Enums

Assembly: libqmp.dll

BlockDeviceIoStatus (Since: 1.0) An enumeration of block device I/O status.

```
public enum BlockDeviceIoStatus : byte
```

Fields

`[JsonPropertyName("failed")] Failed = 1`

The last I/O operation has failed

`[JsonPropertyName("nospace")] Nospace = 2`

The last I/O operation has failed due to a no-space condition

`[JsonPropertyName("ok")] Ok = 0`

The last I/O operation has succeeded